

De las features al diseño arquitectónico

Proyecto Final · AI Data Engineer BSG Institute

En la sesión 1 tu equipo definió el caso de uso: qué problema resolver, qué métricas perseguir y qué fuentes usar. Esta guía te acompaña en el siguiente paso: convertir ese caso en una arquitectura concreta. Como el proyecto del curso tiene un alcance acotado, no construirás todas las features que imaginaste para tu producto. Primero las inventariarás y priorizarás, elegirás una sola a que es el núcleo del caso o la que habilita a las demás y sobre esa feature diseñarás la arquitectura completa por capas.

Cómo usar esta guía

Los recuadros azules contienen un ejemplo resuelto y explican qué se espera en cada campo: léelos como referencia y no los modifiques. Los recuadros en blanco son para que tu equipo complete con la información de su propio proyecto. El ejemplo a lo largo de la guía es un asistente de mesa de ayuda interna basado en RAG, distinto del caso de cotizaciones del material de estudio, para que tengas una segunda referencia.

Parte A · Inventario y priorización de features de IA

El objetivo de esta parte es decidir qué única feature construirás en el curso. No todas las features de tu producto pesan lo mismo: algunas son el corazón del caso de uso, otras son precondition de las demás y otras son mejoras que pueden esperar. Elegir bien aquí evita dispersar el esfuerzo del equipo en algo periférico.

A.1 Inventario de features de IA

Una feature de IA es una capacidad de tu producto que depende de un modelo para funcionar (búsqueda semántica, generación de texto, extracción estructurada, clasificación). Lista solo esas: deja fuera las funciones convencionales como login, reportes o configuración, que no dependen de un modelo. Para cada feature escribe un nombre corto y describe en una frase qué hace y a qué necesidad del usuario responde.

EJEMPLO

#	Feature de IA	Qué hace y a qué necesidad del usuario responde
1	Búsqueda conversacional (RAG)	Responde preguntas del agente con información extraída de la base de conocimiento.
2	Sugerencia de respuesta	Propone un borrador de respuesta al ticket reutilizando la búsqueda en la base.
3	Resumen de tickets largos	Resume el hilo del ticket para que el agente capte el contexto sin leerlo entero.
4	Clasificación y enrutamiento	Asigna categoría y prioridad al ticket y lo dirige al equipo correcto.

TU INVENTARIO

#	Feature de IA	Qué hace y a qué necesidad del usuario responde
1	Extracción estructurada de gastos	Convierte facturas y recibos de proveedores (imprenta, papel, derechos) en datos de gastos tipados para alimentar el sistema.
2	Agente inteligente de determinación de precios	Razona sobre los gastos registrados y políticas de margen para sugerir automáticamente el precio de venta final.
3	Clasificación semántica de costos	Asigna automáticamente categorías contables (costos directos, indirectos, fijos o variables) a cada gasto reportado.
4	Resumen de historial de costos	Genera informes condensados sobre la evolución financiera de un material a través de sus distintas ediciones o reimpresiones.

A.2 Priorización

Evalúa cada feature con cuatro preguntas: cuánto valor entrega al usuario, cuánto esfuerzo técnico exige, si habilita o es precondition de otras features, y si constituye el núcleo del caso de uso. La feature que conviene priorizar es la que es el corazón del caso o la que, al construirse, deja listo el camino para las demás, idealmente con la mejor relación entre valor y esfuerzo. Usa una escala simple de Alto, Medio y Bajo.

EJEMPLO

Feature	Valor	Esfuerzo	¿Habilita otras?	¿Es el núcleo?
Búsqueda RAG	Alto	Medio	Sí (habilita F2)	Sí
Sugerencia de respuesta	Alto	Medio	No	No (depende de F1)
Resumen de tickets	Medio	Bajo	No	No
Clasificación	Medio	Medio	No	No

TU PRIORIZACIÓN

Feature	Valor	Esfuerzo	¿Habilita otras?	¿Es el núcleo?
Extracción estructurada	Alto	Medio	Sí (alimenta al F2)	No
Agente para precios	Alto	Alto	No	Si
Clasificación de costos	Medio	Bajo	No	No
Resumen de rentabilidad	Medio	Bajo	No	No

A.3 Feature priorizada

Enuncia con claridad la feature que tu equipo construirá y justifica en dos o tres frases por qué es el núcleo o el habilitante, conectando con las features que quedan en espera. Esta feature es la única para la que diseñarás arquitectura en la Parte B.

EJEMPLO

Feature priorizada: Búsqueda conversacional sobre la base de conocimiento (RAG).

Es el núcleo del caso porque la propuesta de valor del producto es que los agentes encuentren respuestas confiables sin leer decenas de artículos. Además habilita la feature de sugerencia de respuesta, que reutiliza el mismo retriever. Construir primero la búsqueda RAG deja resuelta la pieza de la que dependen las demás features; el resumen y la clasificación quedan en espera porque aportan menos valor por unidad de esfuerzo.

TU RESPUESTA

Feature priorizada: Agente inteligente para la determinación dinámica de costos y precios de venta.

Es el núcleo del caso de uso porque resuelve directamente el problema identificado: la demora de 2 semanas y el error del 5% en el margen proyectado. Al priorizar el agente, el equipo se enfoca en la capacidad de razonamiento sobre datos operativos, permitiendo que la determinación de precios y costos pase de ser un proceso manual lento a una ejecución casi instantánea. Las otras features, como la extracción de facturas, se consideran complementarias o habilitantes que pueden ser automatizadas posteriormente una vez que el motor de decisión sea robusto.

Parte B · Arquitectura de la feature priorizada

Vas a diseñar la arquitectura únicamente de la feature priorizada, recorriendo las seis capas de referencia. Sigue la secuencia correcta: primero el flujo lógico (qué entra, qué se transforma, dónde se guarda, qué se infiere, cómo se sirve, cómo se observa) y recién después la herramienta concreta de cada capa. Para cada capa indica qué hace en tu feature y con qué herramienta la resolverás, justificando la elección con los criterios del brief: volumen, frescura requerida, costo, privacidad y reversibilidad.

Recordatorio: las seis capas de referencia

- 1 · **Ingesta:** Conectar con las fuentes y traer los datos al pipeline (batch o streaming).
- 2 · **Transformación:** Parsear, normalizar, dividir en chunks y enriquecer con metadatos.
- 3 · **Almacenamiento:** Persistir crudos, estructurados y embeddings en su forma operable.
- 4 · **Inferencia:** Generar embeddings, llamar al modelo de lenguaje y orquestar los pasos.
- 5 · **Serving:** Exponer el sistema a sus consumidores: API, integración o escritura a un sistema.
- 6 · **Observabilidad:** Capa transversal: logging, tracing, feedback, evals y monitoreo de costo.

B.1 Capa de ingesta

Define de qué fuente vienen los datos de tu feature, con qué conector la accederás, en qué modo (carga completa, incremental o captura de cambios) y con qué frecuencia, derivada del SLO del brief.

EJEMPLO

Fuente: artículos de la base de conocimiento alojada en Confluence.
Conector y modo: API REST de Confluence, carga incremental por fecha de modificación.
Frecuencia diaria, suficiente porque la base se actualiza pocas veces al día. Cada artículo se descarga a un bucket de staging.

TU RESPUESTA

Fuente: Registros de gastos de producción almacenados en la base de datos PostgreSQL del sistema editorial.
Conector y modo: Consulta SQL directa a una Materialized View (view_gastos_proyecto). Se utilizará un modo de carga bajo demanda (trigger) disparado cuando el usuario cierra la etapa de registro de gastos en el sistema.

B.2 Capa de transformación

Indica cómo conviertes los datos crudos en algo procesable: con qué herramienta parseas, cómo normalizas, qué estrategia y tamaño de chunking usas (si tu feature lo requiere) y qué metadatos adjuntas para filtrar y auditar después.

EJEMPLO

Parseo y limpieza: extracción del contenido HTML con trafilatura; normalización de codificación y eliminación de navegación y pies repetidos.
Chunking: por estructura (encabezados y párrafos), tamaño objetivo de 400 tokens con 15% de solape. Enriquecimiento con metadatos: categoría, producto, fecha de actualización e identificador del artículo de origen.

TU RESPUESTA

Parseo y limpieza: Al ser datos estructurados provenientes de SQL, el parseo consiste en la validación de tipos mediante Pydantic para asegurar que los montos y fechas sean válidos. Se aplicará una normalización Unicode a las descripciones de los gastos para eliminar caracteres invisibles.
Chunking y metadatos: No se requiere chunking semántico tradicional (RAG), ya que se procesa el listado completo de gastos como un solo contexto de razonamiento por proyecto. Se adjuntarán metadatos de trazabilidad: idproyecto, timestamp_extraccion y version_esquema.

B.3 Capa de almacenamiento

Decide dónde persisten los datos crudos, dónde los metadatos y chunks textuales, y dónde los embeddings. Justifica la elección del vector store con el volumen esperado (número de chunks) y la concurrencia (consultas por segundo).

EJEMPLO

Crudos: bucket de object store para el export de artículos, conservados para reprocesar si cambia la estrategia de chunking.

Estructurado y vectores: PostgreSQL para metadatos y chunks; embeddings en pgvector dentro del mismo Postgres, porque el corpus es de unos 40 mil chunks con baja concurrencia y no justifica un vector store especializado.

TU RESPUESTA

Crudos: Los registros originales permanecen en las tablas transaccionales de PostgreSQL.

Estructurado: Se utilizará una tabla de Logs de Inferencia en PostgreSQL para persistir el JSON de salida del agente, permitiendo auditorías financieras posteriores. Al ser un agente de razonamiento sobre datos numéricos, no se requiere un Vector Store especializado en esta fase inicial.

B.4 Capa de inferencia

Especifica el modelo de embedding, el modelo de generación y la orquestación. Justifica cada elección con calidad para la tarea, costo, latencia y restricciones de privacidad. Recuerda la regla: empieza con la orquestación más simple que cumpla y agrega complejidad solo cuando el flujo lo exija.

EJEMPLO

Embeddings: text-embedding-3-small por buena relación costo-calidad. Generación: Claude Sonnet por seguimiento de instrucciones y salida confiable, vía API.

Orquestación: una función Python simple (recuperar chunks relevantes, armar el prompt con contexto, generar la respuesta). No se usa un framework de agentes porque el flujo es lineal y de un solo paso de generación.

TU RESPUESTA

Modelos: Se utilizará un modelo de alto razonamiento como Gemini 3,1 pro o Gemini 3.5 Flash vía API, debido a la necesidad de precisión matemática y lógica sobre políticas de margen.

Orquestación: Se implementará un Agente (ejecución agéntica) utilizando LangChain o LangGraph que reciba el listado de gastos, identifique categorías y aplique las reglas de negocio del archivo auxiliar JSON para calcular el precio final.

B.5 Capa de serving

Define cómo se expone la feature a quien la consume: una API REST, una integración embebida vía SDK, o la escritura directa a un sistema operacional. La elección depende del patrón de consumo de tu caso.

EJEMPLO

API REST con FastAPI desplegada en un servicio serverless (Cloud Run), consumida por el widget de chat del portal interno de soporte.

Cada solicitud envía la pregunta del agente y recibe la respuesta con las fuentes citadas para que el agente pueda verificarlas.

TU RESPUESTA

Exposición: El sistema se expondrá mediante una API REST (FastAPI) alojada en Cloud Run.

Consumo: El sistema editorial enviará el idproyecto y recibirá un objeto JSON estructurado con el costo propuesto, el precio de venta sugerido y un score de confianza.

B.6 Capa de observabilidad

Esta capa atraviesa todas las demás y suele subestimarse. Indica cómo registrarás cada llamada (input, output, modelo, tokens, costo, latencia), cómo capturarás feedback del usuario y cómo ejecutarás evals periódicos contra el eval set de la sesión 1 para detectar deriva.

EJEMPLO

Logging y tracing: Langfuse captura cada consulta con su prompt, contexto recuperado, tokens, costo y latencia.

Feedback y evals: el agente marca cada respuesta como útil o no útil; un eval set se ejecuta semanalmente con Ragas y dispara una alerta si la fidelidad de las respuestas cae bajo el umbral acordado.

TU RESPUESTA

Logging y tracing: Integración total con Langfuse para registrar cada traza de razonamiento del agente, incluyendo el prompt enviado, la respuesta del modelo y el costo en tokens.

Evals: Se configurará una ejecución automática diaria del Eval Set de 200 casos para detectar desviaciones (drift) en la precisión de los cálculos financieros superiores al 0.5%.

Parte C · Decisión crítica y siguiente paso

Antes de cerrar, identifica la decisión arquitectónica más difícil de revertir de tu feature priorizada: por ejemplo el proveedor del modelo de generación, el vector store, o la disyuntiva entre API y modelo local. Esa será la base del primer Architecture Decision Record (ADR) que escribirás. Anota la decisión, la alternativa que descartaste y la condición bajo la cual la reevaluarías.

EJEMPLO

Decisión: usar pgvector dentro de PostgreSQL en lugar de un vector store especializado.

Alternativa descartada: Pinecone. Se descartó porque el corpus es pequeño y la concurrencia baja, y un motor adicional sumaría operación sin valor demostrable. Reevaluaríamos si el corpus supera el orden de cientos de miles de chunks o si la latencia de búsqueda deja de cumplir el SLO.

TU RESPUESTA

Decisión: Utilizar un modelo de lenguaje de razonamiento avanzado vía API (ej. Gemini 3.1 Pro) en lugar de un modelo local más pequeño.

Alternativa descartada: Modelos locales tipo Llama-3-8B. Se descartaron porque, aunque son más económicos, carecen de la precisión lógica necesaria para tareas contables complejas y el SLO de latencia (60s) permite el uso de modelos más robustos.

Condición de reevaluación: Si el volumen de proyectos aumenta a miles por día y el costo por transacción supera el presupuesto de 0.50 USD.

Con la feature priorizada, su arquitectura por capas y esta decisión crítica documentadas, tu equipo tiene la base para construir el diagrama arquitectónico, el roadmap de fases y el conjunto de ADRs que son el entregable de la sesión 2.